JDBC

JDBC

JDBC

- Java DataBase Connectivity
- API Java permettant
  - la connexion avec les bases de données (e.g. SQLite, MySQL)
  - l'envoi de requête
  - le traitement des résultats

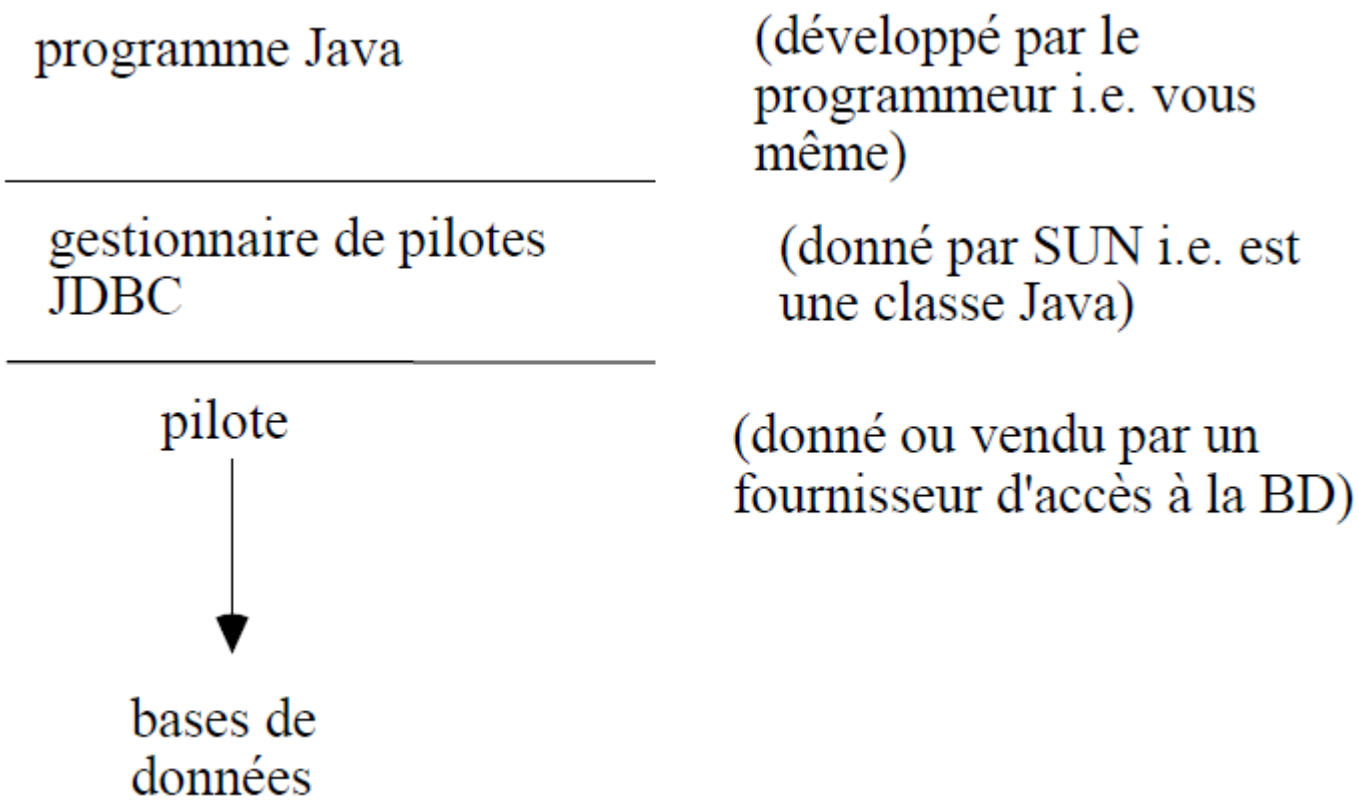
- **Permet de se connecter de la même manière avec la plupart des systèmes de Gestion de Base de Données.**

- Une des grosses difficultés tient plus à la préparation de la base de données pour être utilisée avec JDBC

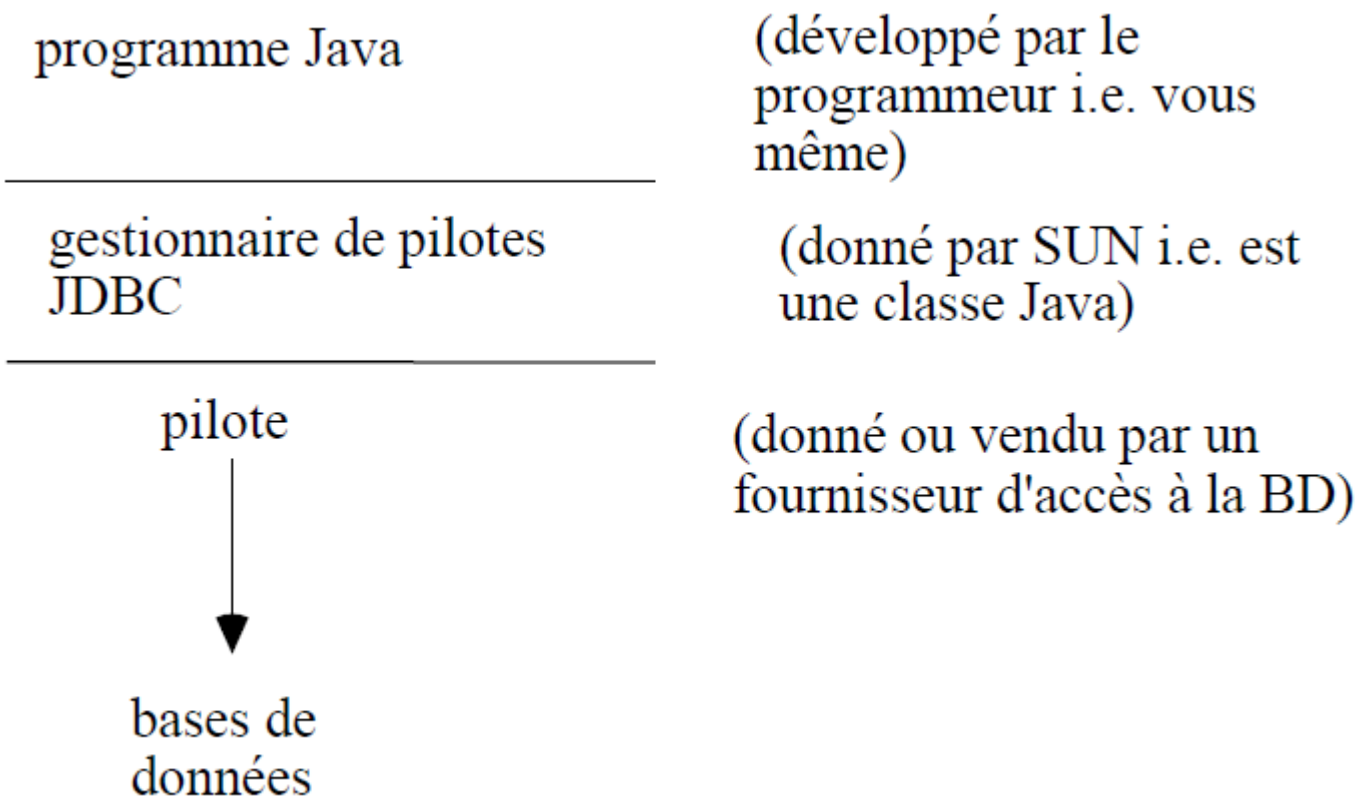
JDBC

- Java数据库连接技术
- 支持 Java API 的应用程序
  - 与数据库（如SQLite、MySQL）的连接
  - 发送请求
  - 结果处理
- 支持与大多数数据库管理系统以相同方式连接。
- 其中一个主要困难在于数据库的准备工作，以便与JDBC配合使用。

ARCHITECTURE DE JDBC



ARCHITECTURE DE JDBC



PRÉPARATION DE L'ENVIRONNEMENT

- Vous avez besoin d'un connector !
  - Soit télécharger un .jar et l'ajouter dans le projet Java (classpath)
  - Soit utiliser MAVEN : ajouter la bonne dependence dans le pom.xml

- Exemple, pour SQLite :

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
</dependency>
```

PRÉPARATION DE L'ENVIRONNEMENT

- 您需要一个连接器！
  - 可下载jar文件并将其添加至Java项目（classpath）中
  - 建议使用MAVEN：在pom.xml文件中添加正确的依赖项

- 以SQLite为例：

```
<依赖性>
        （组别）.xerial（/组别）
        (artifactId)sqlite-jdbc(/ artifactId)</dependency>
```

EXEMPLE 0 : SE CONNECTER À UNE BASE DE DONNÉES

示例0：连接数据库

4 ÉTAPES

1. Créer un objet **Connection** qui représente la connexion au serveur de BD
2. Créer un ou plusieurs objets **Statement** pour manipuler la BD
3. Utiliser les objets **Statement** pour manipuler la BD
4. Fermer la connexion à la BD

4 ÉTAPES

1. 创建一个Connection对象，用于建立与数据库服务器的连接
2. 创建一个或多个语句对象以操作数据库
3. 使用Statement对象操作数据库
4. 关闭与数据库的连接

1. Créer un objet Connection qui représente la connexion au serveur de BD

- **L'URL du serveur de BD**

```
String url = "jdbc:sqlite:./db/conferences.db";
```

```
String urlMySQL = "jdbc:mysql://localhost:3306/mysql";
```

- **Créer une connexion à la BD**

```
Connection con = DriverManager.getConnection(url);
```

```
Connection conMySQL = DriverManager.getConnection(urlMySQL,"root", "");
```

  - Si la tentative échoue, une exception de type **SQLException** sera levée. L'information pertinente à la nature du problème sera encapsulée dans l'objet SQLException

1. CRÉER UN OBJET CONNECTION QUI REPRÉSENTE LA CONNEXION AU SERVEUR DE BD

- 数据库服务器的URL

```
String url = "jdbc:sqlite: . /db/conferences . db" ;
```

```
String urlMySQL = "jdbc:mysql://localhost:3306/mysql";
```

- 创建与数据库的连接

```
连接方式为: DriverManager.getConnection（网址）;
```

```
连接 conMySQL = DriverManager .getConnection(urlMySQL, "root" , "" );
```

  - 若操作失败，系统将抛出SQLException异常。与问题性质相关的详细信息将被封装在SQLException对象中。

2.CRÉER UN OBJET STATEMENT

- Un objet **Connection** définit une connexion (session) avec un BD spécifique.

- Des instructions (statement) SQL sont exécutées et des résultats sont retournés dans le contexte d'une connexion.

- Un objet **Statement** est utilisé pour exécuter une/plusieurs instructions SQL et pour retourner les résultats qu'elle(s) produi(sen)t

```
Statement stmt;
stmt = con.createStatement();
```

2.CRÉER UN OBJET STATEMENT

- **Connection对象用于建立与特定数据库的连接（会话）。**

- **SQL语句在连接上下文中执行并返回结果。**

- **Statement对象用于执行一条或多条SQL语句，并返回生成的结果。**

```
声明语句;
语句：con . createStatement();
```

3. MÉTHODES DE STATEMENT

- **Statement** possède de nombreuses méthodes qui permettent d'accéder à la BD et de manipuler ses données

- **`int executeUpdate(String SQLcomm)`**
  SQLcomm doit être une commande SQL valide
  - Cette méthode est utilisée pour exécuter les instructions SQL INSERT, UPDATE ou DELETE
  - Les autres instructions SQL qui ne retournent rien peuvent être exécutées en utilisant cette méthode

- **`ResultSet executeQuery(String sql)`**
  - Exécute une requête SQL sql qui renvoie un résultat sous forme de ResultSet.
  - Utilisé pour les instructions de type SELECT

3. MÉTHODES DE STATEMENT

- **Statement**拥有多种方法，可用于访问数据库并操作其中的数据

- **`int executeUpdate(String SQLcomm)`**
  *SQLcomm*必须是有效的 SQL 命令
  - 该方法用于执行SQL语句中的INSERT、UPDATE或DELETE操作。
  - 其他不返回任何结果的SQL语句可通过此方法执行

- **ResultSet执行查询（String sql）**
  - 执行SQL查询，返回ResultSet类型的结果。
  - 用于SELECT语句类型

4.FERMER LA CONNEXION

```
con.close();
```

4.FERMER LA CONNEXION

```
con . close();
```

RÉSUMÉ

```
import java.sql.Connection;
import java.sql.Statement;

public class prog1 {

  public static void main(String args[]){
   try {
      Statement stmt;
      String url = "jdbc:sqlite:./db/santa.db";
      Connection con = DriverManager.getConnection(url);
      stmt = con.createStatement();
      // ……————————————.
      con.close();
   } catch( Exception e ) {  e.printStackTrace(); }//end catch
  }//end main
}
```

RÉSUMÉ

```
导入java.sql Connection；
import    java.sql    Statement；public

class prog1 {

  public static void main(String args[]){
   try  {
        声明语句;
        String url =  “jdbc:sqlite: . /db/santa . db”  ;
        连接方式为:  DriverManager.getConnection（网址）;
        语句:  con . createStatement();
        //  ..........................
        con . close();

    }catch( Exception e ) { e .printStackTrace();  }//end catch}//end main

}
```

RÉSUMÉ (VERSION AVEC GESTION DES RESSOURCES DANS LE TRY)

```
import java.sql.Connection;
import java.sql.Statement;

public class prog1 {

  public static void main(String args[]){
   try {
     String url = "jdbc:sqlite:./db/santa.db";
     try (Connection conn = DriverManager.getConnection(url)) {
       System.out.println("Connection to SQLite has been established. »);
       Statement stmt;
       stmt = con.createStatement();
      // ……————————
    } catch (SQLException e){  e.printStackTrace(); }//end catch
  }//end main
}
```

RÉSUMÉ (VERSION AVEC GESTION DES RESSOURCES DANS LE TRY)

```
导入java.sql Connection；
import    java.sql    Statement；public

class prog1 {

  public static void main(String args[]){
   try  {
      String url =  “jdbc:sqlite: . /db/santa . db”   ;
      try(Connection conn = DriverManagergetConnection(url)) {
         系统输出：System.out.println(“已建立与SQLite的连接。”);  语句：stmt;
         语句：con . createStatement();
        //  ....................
     } catch (SQLException e){ e .printStackTrace()；  }//end catch}//end main
}
```

EXEMPLES

Manipuler la base et ses tables

示例

操作数据库及其表

CRÉATION DE LA TABLE JOUETS

```
stmt.executeUpdate(
"CREATE TABLE IF NOT EXISTS Jouets(
        jouet_id INTEGER PRIMARY KEY AUTOINCREMENT,
        jouet_nom text NOT NULL);");
```

- La nouvelle table aura deux colonnes
  - **jouet_id** : un entier
  - **jouet_nom** : une chaîne de 15 caractères max

CRÉATION DE LA TABLE JOUETS

语句执行更新操作（
若表 Jouets 不存在则创建该表
玩具ID（jouet_id）为整型主键，采用自动递增机制；玩具名称（jouet_nom）为文本类型且不能为空；

- 新表格将包含两列
  - **jouet_id**：一个整数
  - **jouet_nom**：最长15个字符的字符串

INSÉRER DES VALEURS DANS LA TABLE

- **jouet_id** est auto-incrémenté, donc pas besoin de le préciser
- « Playmobil » est inséré dans la colonne **jouet_nom**

```
stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES('Playmobil')");
```

INSÉRER DES VALEURS DANS LA TABLE

- **jouet_id**采用自动递增机制，因此无需特别说明
- **“Playmobil”被插入到玩具名称列中**

语句执行更新操作： “INSERT INTO Jouets(jouet_nom) VALUES(‘Playmobil ‘)” ;

INSERTION DE 4 LIGNES SUPPLÉMENTAIRES

```
stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES(’Velo')");

stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES(’Poupee')");

stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES(’Orange')");

stmt.executeUpdate("INSERT INTO Jouets(jouet_nom) VALUES(’Balai')");
```

INSERTION DE 4 LIGNES SUPPLÉMENTAIRES

```
stmt .executeUpdate( “INSERT INTO Jouets(jouet_nom) VALUES（ ‘ 自 行 车 ‘）” ); stmt .executeUpdate
( “INSERT INTO Jouets(jouet_nom) VALUES（ ‘洋娃娃’ ）” ); stmt .executeUpdate( “INSERT INTO Jouets
(jouet_nom) VALUES（ ‘橙子 ‘）” ); stmt .executeUpdate( “INSERT INTO Jouets(jouet_nom) VALUES（ ‘扫
帚’ ）” );
```

INSERTION DE 4 LIGNES SUPPLÉMENTAIRES

- On peut aussi ajouter des requêtes incomplètes (ou paramètrées) avec **PreparedStatement**.

```
List<String> jouets = List.of("Velo","Poupee","Orange","Balai");

 String sql = "INSERT INTO Jouets(jouet_nom) VALUES(?)";
 PreparedStatement pstmt = conn.prepareStatement(sql);

 for(String j : jouets){
    pstmt.setString(1,j);
    pstmt.executeUpdate();
 }
```

INSERTION DE 4 LIGNES SUPPLÉMENTAIRES

- 还可以通过PreparedStatement添加不完整的查询（或带参数的查询）。

```
List<String> jouets = List.of("Velo","Poupee","Orange","Balai");

字符串sql= “INSERT INTO玩具(jouet_nom)VALUES（？）” ;
pstmt=conn.prepareStatement(sql);

for(String j : jouets){
    pstmt.setString(1,j);
    pstmt. executeUpdate();
}
```

INTERROGATION DE LA BASE

- Jusqu'à présent nous n'avons utilisé que la méthode **executeUpdate** de l'interface **Statement**

- Rappel : executeUpdate peut être utilisée pour exécuter des commandes SQL qui ne retournent aucun résultat

- Pour exécuter une commande SQL qui retourne des résultats nous utiliserons la méthode **executeQuery** sur l'objet **Statement**

INTERROGATION DE LA BASE

- 到目前为止，我们仅使用了Statement接口中的executeUpdate方法。

- 注意：executeUpdate可用于执行不返回结果的SQL语句

- 要执行返回结果的SQL语句，我们需要对Statement对象调用executeQuery方法。

RAPPEL : MÉTHODES DE STATEMENT

- **ResultSet executeQuery(String sql)**
  - Exécute une requête SQL sql qui renvoie un résultat sous forme de **ResultSet**.

- Exception : SQLException
  Si :
  - l'acces à la base de données provoque une erreur
  - Si executeQuery est appelé sur une Statement fermée
  - Si la requête produit autre chose qu'un ResultSet

RAPPEL : MÉTHODES DE STATEMENT

- **ResultSet执行查询（String sql）**
  - 执行SQL查询，返回ResultSet类型的结果。

- 异常：SQLException
  是：
  - 数据库访问时出现错误
  - 当在已关闭的语句上调用executeQuery方法时
  - 如果查询结果不是ResultSet类型

UN OBJET RESULTSET

- Un objet ResultSet fourni un accès à une table de données encapsulée
- L' object maintient un curseur pointant sur la ligne de données courante
- Initialement le curseur est positionné **avant** la première ligne

- La méthode **next()** de ResultSet bouge le curseur à la ligne suivante (similaire à un itérateur ou un énumérateur en Java)
- La méthode next() renvoie false s'il n'y a pas de ligne suivante.

UN OBJET RESULTSET

- **ResultSet**对象可访问封装的数据表
- 该对象保持一个光标指向当前数据行
- 初始时，光标位于第一行之前。
- **ResultSet**的**next()**方法会将游标移动到下一行（类似于**Java**中的迭代器或枚举器）
- 若无下一行，则**next()**方法将返回**false**。

MANIPULATION D'UN RESULTSET

```
stmt = con.createStatement();

ResultSet rs = stmt.executeQuery(
                    "SELECT * FROM Jouets ORDER BY jouet_id");

while(rs.next()){

}//end while
```

MANIPULATION D'UN RESULTSET

语句：con . createStatement();

```
ResultSet rs = stmt.executeQuery(
                                SELECT * FROM Jouets ORDER BY jouet_id

while(rs.next()){

} //while循环结束
```

MÉTHODES D'UN OBJET RESULTSET

- Une fois que les résultats sont encapsulés dans un objet ResultSet, l'interface ResultSet fournit plusieurs méthodes qui permettent d'extraire les informations de l'objet

- Les méthodes *get** (comme le *getString*) retourne les valeurs des colonnes pour la ligne courante :
  - `getString(String columnLabel)`
  - `getString(int columnIndex)`

- On peut retourner les valeurs soit en utilisant l'indice de la colonne soit son nom.
- Note: les colonnes sont numérotées **à partir de 1 pas de 0**
- Le pilote JDBC gère en principe la conversion des données en type Java

MÉTHODES D'UN OBJET RESULTSET

- 当查询结果被封装到ResultSet对象中后，ResultSet接口会提供多种方法，用于从该对象中提取信息。
- *get**方法（例如*getString*）会返回当前行中各列的数值：
  - 获取字符串（字符串列标签）
  - 获取整型数据（通过索引列）
- 可以通过列索引或列名称来返回值。
- 注：各列编号从1开始，不包括0
- JDBC驱动程序主要负责数据向Java类型进行转换

EXEMPLE

```
stmt = con.createStatement();

ResultSet rs = stmt.executeQuery("SELECT * FROM Jouets ORDER BY jouet_id");

System.out.println("Liste des cadeaux :");

while(rs.next()){
   int monJouetId= rs.getInt("jouet_id");
   String nomJouet = rs.getString("jouet_nom");
   System.out.println("\tid= " + monJouetId + "\tnom = " + nomJouet);
}//end while
```

Une SQLException est levée par get****() si :

- columnIndex n'est pas valide
- Erreur d'accès à la BD ou ResultSet fermé

EXEMPLE

语句：con . createStatement();

ResultSet rs = stmt .executeQuery(“SELECT * FROM Jouets ORDER BY jouet_id”);

```
System.out.println("Liste des cadeaux :");

while(rs.next()){
    int monJouetId= rsgetInt( “玩具ID” );
    String nomJouet = rs.getString("jouet_nom");
    系统。输出.println( “\tid= ” + monJouetId + “\tnom = ” + nomJouet； ) //while 结束
```

当满足以下条件时，get****()方法将抛出SQLException异常：

- columnIndex 不是有效值
- 无法访问数据库或已关闭的结果集（ResultSet）

SORTIE

Liste des cadeaux:

id= 1 nom = Playmobil

id= 2 nom = Velo

id= 3 nom = Poupee

id= 4 nom = Orange

id= 5 nom = Balai

SORTIE

礼 品 清 单 ： id=1 名 称 =

Playmobil

id= 2 nom = 车轮
id= 3 nom = 玩偶
id=4 nom=橙色
id= 5 nom = Balai

LA CLASSE RESULTSETMETADATA

- Un objet de type ResultSetMetaData permet d'avoir des information sur le type et les propriétés des colonnes d'un objet ResultSet.

```
String query = "SELECT * FROM Jouets";
ResultSet results = stmt.executeQuery(query);

ResultSetMetaData rsmd = results.getMetaData();
int numcols = rsmd.getColumnCount();

for(int i = 1;i<=numcols;i++){
    System.out.println(rsmd.getColumnName(i));
 }
```

LA CLASSE RESULTSETMETADATA

- **ResultSetMetaData对象可用于获取ResultSet对象中列的类型及属性信息。**

```
String query = "SELECT * FROM Jouets";
ResultSet 结果 = stmt执行查询语句;

ResultSetMetaData rsmd =results.getMetaData(); int numcols =
rsmd.getColumnCount();

for(int i = 1;i<=numcols;i++){
    系统。输出.println(rsmd . getColumnName(i)); }
```

AFFICHER LES DONNÉES DE LA LIGNE 2

Affichage de la ligne 2 :

id= 2 nom = Velo

```
System.out.println("Affichage de la ligne 2 :");
if( rs.absolute(2) ){
  int monJouetId= rs.getInt("jouet_id");
  String nomJouet = rs.getString("jouet_nom");
  System.out.println("\tid= " + monJouetId  + "\tnom = " +
nomJouet);
}//end if
```

AFFICHER LES DONNÉES DE LA LIGNE 2

第2行显示：id=2 名称=自

行车

```
System.out.println("Affichage de la ligne 2 :");
if( rs.absolute(2) ){
  int monJouetId= rs.getInt("jouet_id");
  String nomJouet = rs.getString("jouet_nom");
  系统。输出.println( "\tid= " + monJouetId + "\tnom = " + nomJouet);
//结束if语句
```